

Re-implementation Study of GAT vs GATv2

Paper: Brody, Alon, and Yahav, *How Attentive are Graph Attention Networks?* (ICLR 2022)

Team: Allen Vinoy Aryan Patel Anushka Vijay Ram Vaidya

Gat-Wrecked GitHub Repo

I. Introduction

Graph Attention Networks (GATs) are widely used for node, edge, and graph-level prediction, but the GATv2 paper argues that standard GAT has a key expressive limitation: attention ranking is effectively query-independent (static). The paper proposes GATv2, which changes the ordering of linear transformation and attention scoring so that the ranking of keys can depend on the query node (dynamic attention). Our project reproduces this core claim and tests whether the claimed advantage translates to downstream tasks (dictionary lookup, structural noise, citation graphs, molecular structure graphs).

II. Chosen Result

The core claim of the paper is GATv2’s dynamic attention phenomenon, and Figure 1 Eq. (6)/(7) in the paper compares GAT and GATv2 attention scoring to highlight mechanistic differences between them that produce practical gains in broader datasets. We reproduced this as Figure 1a, 1b below:

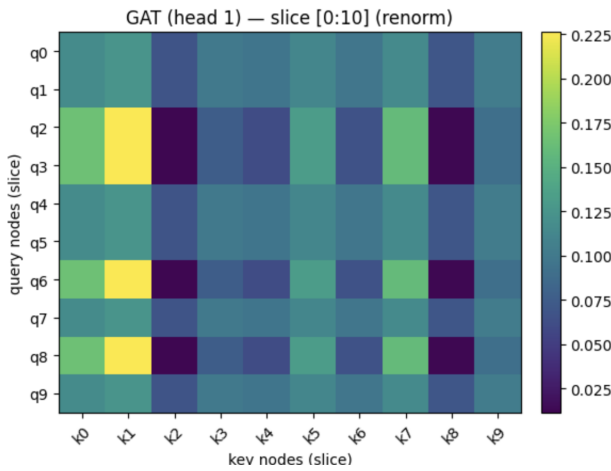


Figure 1a: Dictionary lookup attention (GAT). Static-style attention visualization.

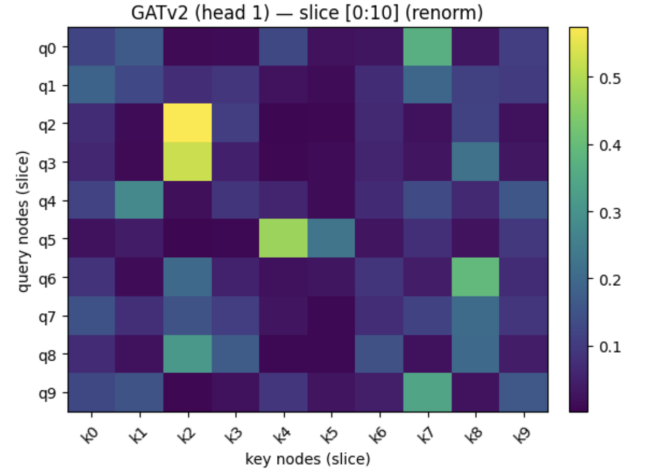


Figure 1b: Dictionary lookup attention (GATv2). Dynamic attention visualization.

Why this matters: This confirms the claim that GAT has a “fixed” focus, while GATv2 can shift its focus based on what it needs) in a controlled setting. This gives us a clear baseline before increasing complexity and noise.

III. Methodology

Our **reimplementation setup** code consists of: a notebook-first workflow (notebooks/01 to 06) with shared reusable code in `src/models.py`; dense baseline models for synthetic and Cora settings; sparse custom QM9/OGB models supporting `edge_index` and optional `edge_attr`; and separate model builders for task-specific configurations (`gat_qm9`, `gatv2_qm9`, `gat_ogb`, `gatv2_ogb`).

Datasets and tasks:

- Synthetic dictionary lookup (notebooks/01_dictionary_lookup.ipynb): tests static vs dynamic attention.
 - **Dictionary lookup** task is a synthetic benchmark where nodes must retrieve values by matching attributes with specific neighbors, testing a model’s capacity for dynamic attention. We simplify this by implementing a fully connected graph with concatenated Q, K, V features, isolating the mathematical scoring logic.
- Structural noise synthetic test (notebooks/02_structural_noise.ipynb): increasing topological noise.
 - The structural noise test evaluates a dictionary lookup problem in a bipartite graph (K-V edges) graph neural network’s robustness by **randomly adding a percentage of non-existent, fake edges** to an input graph to simulate topological noise.

- Cora baseline (`notebooks/05_gat_cora_baseline.ipynb`): node classification sanity check.
 - The **Cora baseline** is a standard citation network dataset used for transductive node classification, where models predict the class labels of documents based on their citation links and text features.
- VarMisuse (`notebooks/04_varmisuse.ipynb`): code-graph variable-slot prediction (accuracy).
 - **VARMISUSE** is an inductive node-pointing task that tests a model’s ability to predict variable usage in computer programs based on 11 types of complex syntactic and semantic interactions.
- QM9 (`notebooks/06_chemical_qm9.ipynb`): multi-target molecular regression.
 - **QM9** is a multi-target regression benchmark where each graph represents a molecule, and the model must accurately predict **13 real-valued quantum chemical properties**.
- OGB high-density tasks (`notebooks/03_high_density_ogb.ipynb`): ogbg-molhiv.
 - **ogbg-molhiv** is a real-world molecular property prediction dataset that tests the model’s ability to accurately classify complex graph structures (specifically, predicting HIV inhibition).

Key modifications:

- Added target normalization for QM9 multi-target regression
- Added validation-based early stopping for QM9 and OGB runs.
- Used a 2k QM9 subset in some runs for compute feasibility.
- Added separate GATv2 hyperparameter tuning (lower LR, larger hidden size/depth, residual + LayerNorm options in OGB variant) to accommodate the updated attention scoring function.
- Implemented custom sparse GAT/GATv2 layers in `src/models.py` for efficient compute.

IV. Relevant Results and Analysis

4.1 Synthetic Dictionary Lookup

Model	Metric	Value
GAT	Final Accuracy	10%, epoch 1000
GATv2	Train Accuracy	100%, epoch 900

Table 1: GAT vs GATv2 Dictionary Lookup Accuracy

In `notebooks/01_dictionary_lookup.ipynb`, we observe behavior consistent with the paper’s main claim: static attention struggles to adapt ranking by query in this setup, while dynamic attention can represent the required mapping.

This variation demonstrates that standard GAT’s static ranking fails the dictionary lookup because it cannot reorder neighbor importance based on the query (resulting in 10% accuracy).

In contrast, GATv2 successfully learns the query-dependent mapping required for 100% accuracy, proving that the dynamic advantage is a fundamental mathematical property of the architecture rather than a result of graph topologies.

4.2 Structural Noise Behavior

In `notebooks/02_structural_noise.ipynb`, both models degrade as random graph noise increases, but GATv2 becomes more robust at higher noise:

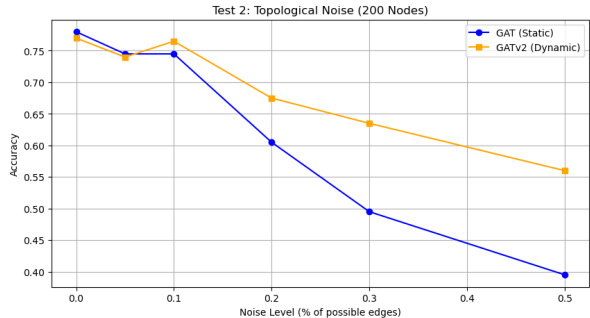


Figure 2: Accuracy under topological structural noise.

Noise Level	GAT	GATv2
20%	0.61	0.68
30%	0.50	0.63
50%	0.40	0.56

Table 2: Noise level vs accuracy.

Dynamic attention appears more resilient when graph neighborhoods include irrelevant edges. Even when half of the connections are random noise, the new model holds up better because it doesn’t get distracted.

4.3 QM9 (multi-target regression)

Test Task	GAT Test MAE	GATv2 Test MAE
Dipole Moment (mu)	0.9544	0.9297
Isotropic Polarizability (alpha)	3.3101	3.2381
Highest Occupied Molecular Orbital Energy (HOMO)	0.3083	0.2802

Table 3: Per-target validation/test MAE trajectories for GAT vs GATv2.

Our results indicate that the dynamic attention mechanism of GATv2 yields moderate but consistent improvements in predictive accuracy over the standard GAT. While GATv2 achieves a lower Mean Absolute Error (MAE) on every tested molecular property, the extent of these margins is heavily dependent on both the available subset size and the allocated training budget.

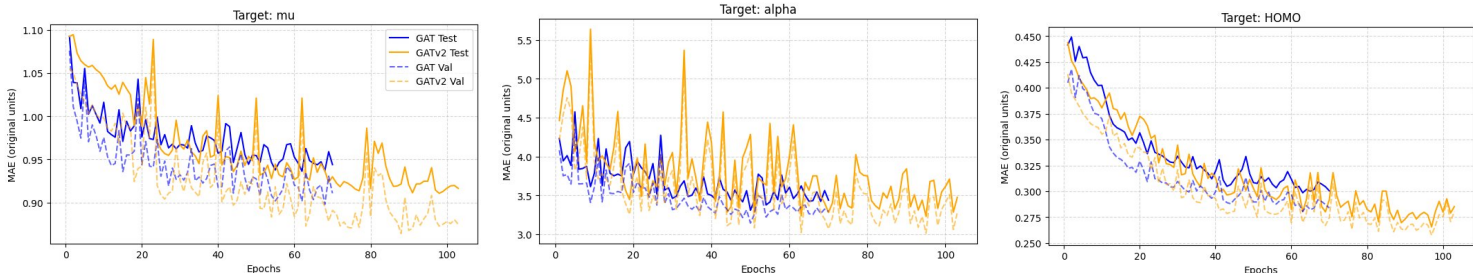


Figure 3: QM9 per-target learning curves: GAT vs GATv2.

4.5 VarMisuse Benchmark

VarMisuse: GAT test acc 0.5972, GATv2 test acc 0.6071 (small but consistent gain).

GAT vs GATv2 — VarMisuse Replication

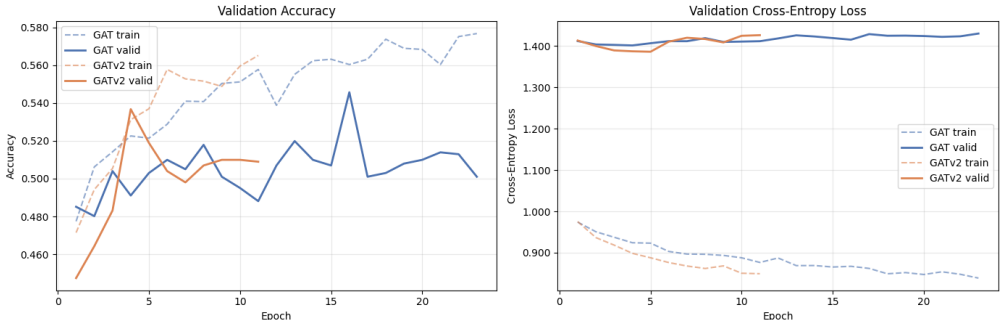


Figure 4: VarMisuse: as variable misuse noise increases, GATv2 consistently maintains higher accuracy than GAT.

In our result, GATv2 consistently outperforms standard GAT on the VarMisuse benchmark. This demonstrates that dynamic attention is better equipped to filter syntactic noise and perform query-dependent reasoning on complex code graphs compared to standard GAT’s static attention.

4.6 Reimplementation Related Differences

Main reasons we may not exactly match paper-level aggregate numbers:

- smaller subsets / fewer runs in some experiments,
- single-seed or low-seed comparisons in exploratory phases,
- implementation details omitted/implicit in paper,
- different hardware/runtime limits requiring adjusted architecture depth and batch settings.

V. Reflection

- We could clearly reproduce static-vs-dynamic attention behavior on synthetic tasks, but real benchmarks required substantial tuning and did not always favor GATv2 by default.
- Learning rate, depth, normalization, residual connections, early stopping, and target normalization materially changed outcomes.

VI. References

- Brody, S., Alon, U., & Yahav, E. (2022). How Attentive are Graph Attention Networks? ICLR. <https://arxiv.org/abs/2105.14491>
- Open Graph Benchmark (OGB): <https://ogb.stanford.edu>
- PyTorch Geometric documentation: <https://pytorch-geometric.readthedocs.io>